SQL CASE STUDY

SWIGGY ORDER & DELIVERY DATA ANALYSIS

MySQL Query-Based Business Insights

curiosity.sql

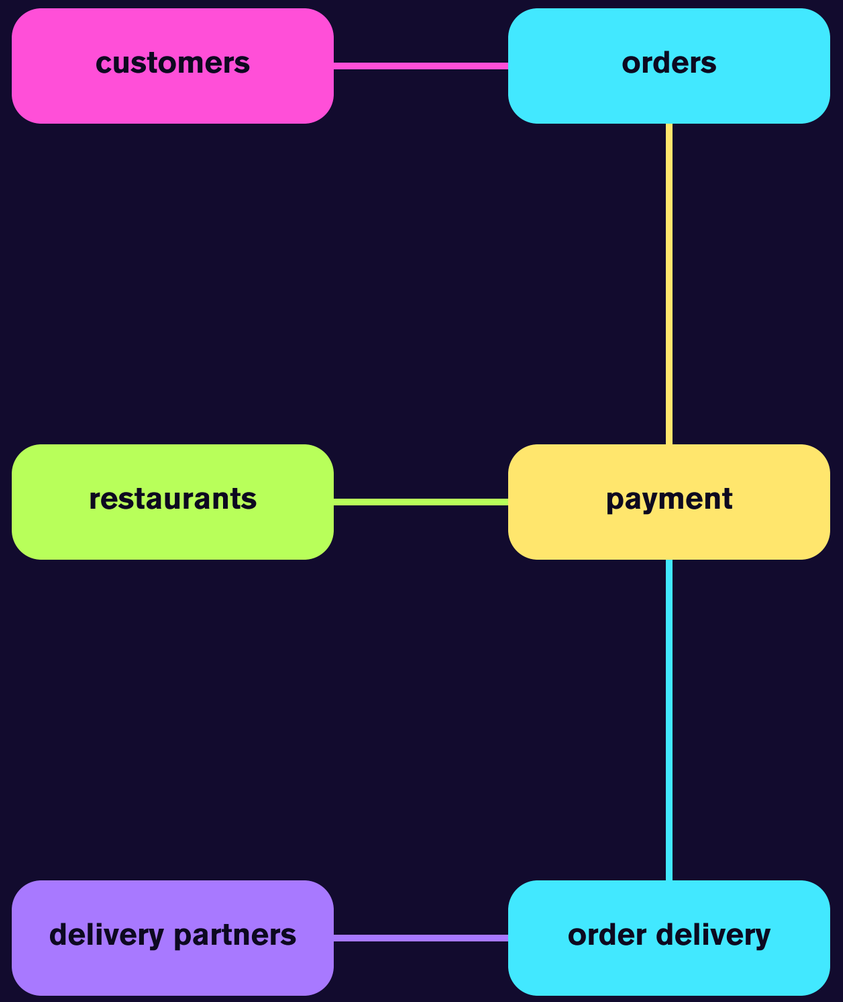
```sql
SELECT insights
FROM swiggy_data
JOIN curiosity
WHERE question = "why"
ORDER BY impact DESC;
```

Presented by Kunal Malviya

Inside Swiggy's Data Kitchen

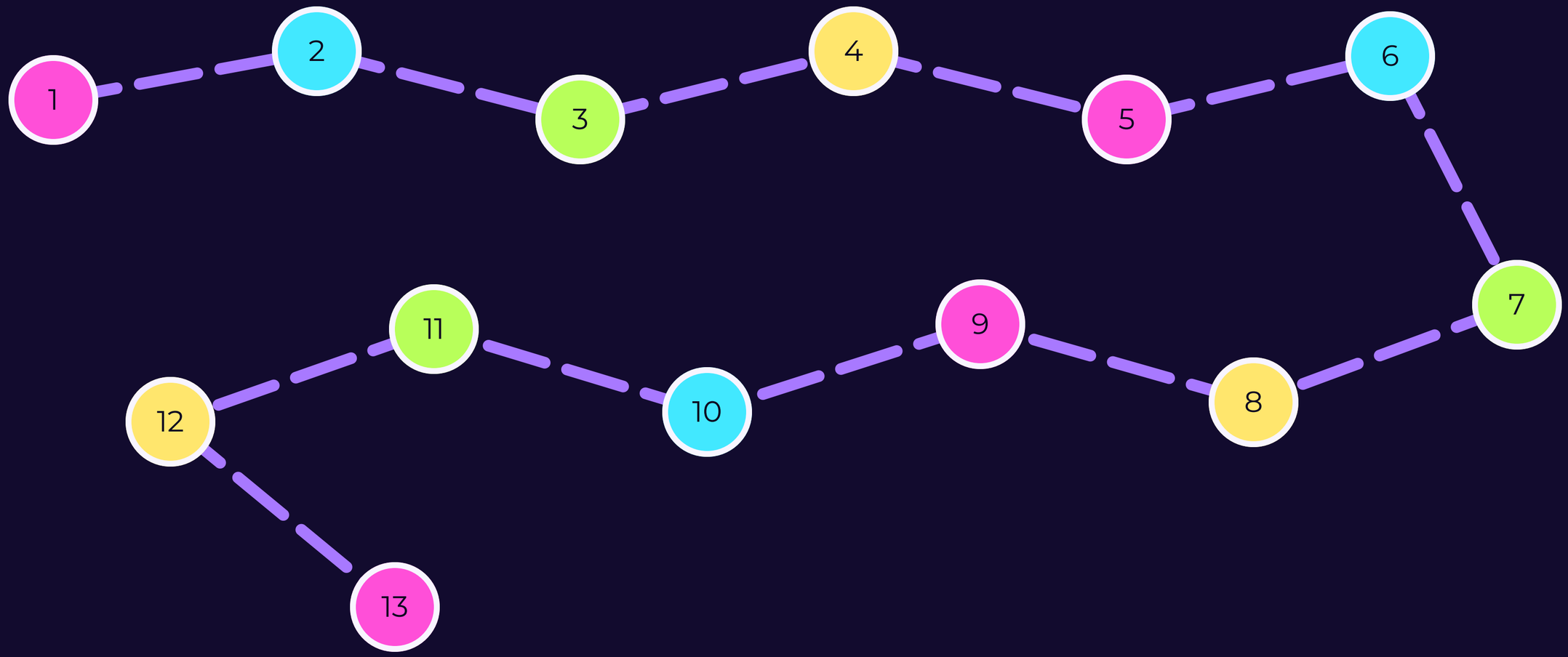
Swiggy wants to turn order and delivery records into smart decisions. This case study answers 13 business questions with MySQL using joins, aggregations, subqueries and CTEs.

6 TABLES • 13 QUESTIONS



The 13 Questions

A winding route from filters to self-joins.



Q1

Display all customers who live in 'Delhi'.

query.sql

```sql
SELECT *
FROM customers
WHERE city = 'Delhi';
```

Q2

Find the average rating of all restaurants in 'Mumbai'.

query.sql

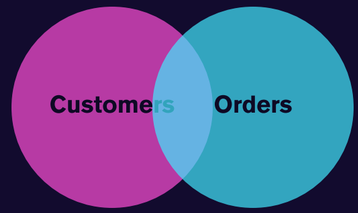
```sql
SELECT AVG(rating) AS average_rating
FROM restaurants
WHERE city = 'Mumbai';
```

Q3

List all customers who have placed at least one order.

query.sql

```sql
SELECT DISTINCT c.*
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```



Q4

Display the total number of orders placed by each customer.

query.sql

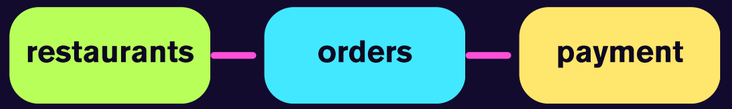
```sql
SELECT c.customer_id, c.customer_name,
       COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name;
```

Q5

Find the total revenue generated by each restaurant.

query.sql

```sql
SELECT r.restaurant_id, r.restaurant_name,
       SUM(p.amount) AS total_revenue
FROM restaurants r
JOIN orders o ON r.restaurant_id = o.restaurant_id
JOIN payment p ON o.order_id = p.order_id
GROUP BY r.restaurant_id, r.restaurant_name;
```

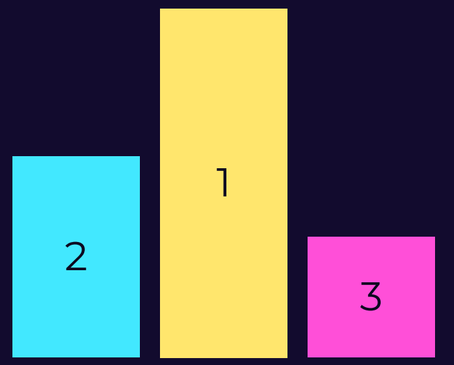


Q6

Find the top 5 restaurants with the highest average rating.

query.sql

```sql
SELECT restaurant_id, restaurant_name,
       AVG(rating) AS average_rating
FROM restaurants
GROUP BY restaurant_id, restaurant_name
ORDER BY average_rating DESC
LIMIT 5;
```



Q7

Display all customers who have never placed an order.

Method 1 — LEFT JOIN

```sql
SELECT c.*
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Method 2 — Subquery

```sql
SELECT *
FROM customers c
WHERE c.customer_id NOT IN (
  SELECT customer_id FROM orders
);
```



Q8

Find the number of orders placed by each customer in 'Mumbai'.

query.sql

```sql
SELECT c.customer_id, c.customer_name,
       COUNT(o.order_id) AS total_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE c.city = 'Mumbai'
GROUP BY c.customer_id, c.customer_name;
```

Q9

Display all orders placed in the last 30 days.

query.sql

```sql
SELECT *
FROM orders
WHERE order_date >=
DATE_ADD(CURRENT_DATE, INTERVAL -30 DAY);
```

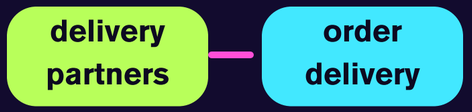
LAST
30 DAYS

Q10

List all delivery partners who have completed more than 1 delivery.

query.sql

```sql
SELECT dp.delivery_partner_id, dp.partner_name,
       COUNT(od.order_id) AS completed_deliveries
FROM delivery_partners dp
JOIN order_delivery od
ON dp.delivery_partner_id = od.delivery_partner_id
WHERE od.delivery_status = 'Completed'
GROUP BY dp.delivery_partner_id, dp.partner_name
HAVING COUNT(od.order_id) > 1;
```



Q11

Find the customers who have placed orders on exactly three different days.

query.sql

```sql
SELECT c.customer_id, c.customer_name
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(DISTINCT DATE(o.order_date)) = 3;
```

DAY 1

DAY 2

DAY 3

Q12

Find the delivery partner who has worked with the most different customers.

query.sql

```sql
SELECT dp.delivery_partner_id, dp.partner_name,
       COUNT(DISTINCT o.customer_id) AS unique_customers
FROM delivery_partners dp
JOIN order_delivery od
ON dp.delivery_partner_id = od.delivery_partner_id
JOIN orders o ON od.order_id = o.order_id
GROUP BY dp.delivery_partner_id, dp.partner_name
ORDER BY unique_customers DESC
LIMIT 1;
```

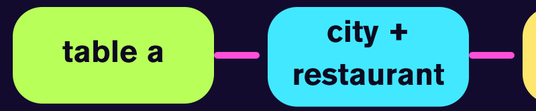


Q13

Identify customers who have the same city and have placed orders at the same restaurants, but on different dates.

query.sql

```sql
SELECT DISTINCT a.customer_id AS customer_1,
       b.customer_id AS customer_2, a.city,
       oa.restaurant_id
FROM customers a
JOIN customers b ON a.city = b.city AND a.customer_id < b.customer_id
JOIN orders oa ON a.customer_id = oa.customer_id
JOIN orders ob ON b.customer_id = ob.customer_id
AND oa.restaurant_id = ob.restaurant_id
AND DATE(oa.order_date) <> DATE(ob.order_date);
```



THANK YOU,

Behind every great decision is a well-asked question. Joins connect the dots, grouping reveals the pattern, and a good query turns raw numbers into a story worth acting on.

KEEP ASKING WHY